# Design and Performance Evaluation of Secure Edge–Cloud Split VLM Inference with TLS 1.3 and Intel TDX

Yaoquan Ma

***Abstract*—Vision-language models (VLMs) require significant memory and computational resources. This makes full deployment difficult on resource-limited edge devices. The model can be divided between the edge device and the cloud server to reduce the workload at the edge. The edge device extracts visual embeddings from the image, and the cloud server completes the remaining inference. Because these embeddings may contain sensitive information, they require protection during transmission and cloud-side processing.**

**In this study, we present and evaluate a secure split VLM inference framework. The framework uses Transport Layer Security (TLS) 1.3 to protect the visual embeddings during transmission. Intel Trust Domain Extensions (TDX) provides protection for the reconstructed embeddings and inference state during cloud-side processing. Remote attestation allows the edge client to verify the TDX execution environment before sending the embeddings. Embedding transfer proceeds only after successful verification.**

**The evaluation used nanoVLM and the AI2D dataset. The split-inference configurations produced outputs consistent with the original model under the evaluated settings. TLS introduced limited transfer overhead relative to the model computation time. Intel TDX increased cloud inference latency and cloud process CPU time by approximately 11%. Further profiling showed that autoregressive decoding was the main source of this overhead. End-of-sequence (EOS) early stopping reduced generation latency by approximately 67%, although the relative TDX overhead remained close to 9%. These results show that the framework provides additional protection with a moderate performance cost for the evaluated workload.**

***Index Terms*—Confidential computing, Intel TDX, intermediate embeddings, remote attestation, split inference, TLS, vision-language models.**

## I. Introduction

VISION-language models (VLMs) have experienced rapid development in recent years. VLMs can understand both text and images, and they can perform tasks based on visual inputs, such as visual question answering, image captioning, and visual reasoning [1], [2]. At the same time, the Internet of Things (IoT) and edge computing have also developed rapidly, creating more demand for intelligent services on edge devices. As a result, VLMs are increasingly becoming useful in many edge application scenarios, such as autonomous driving systems, intelligent monitoring, and real-time visual analysis [2].

Yaoquan Ma is a graduate student with the School of Computer Science, University of Galway, Galway, Ireland (e-mail: Y.Ma10@universityofgalway.ie).

Although VLMs offer powerful image understanding and language reasoning abilities, deploying them directly on edge devices is significantly challenging. Modern VLMs typically consist of a visual encoder, a projection module, and a large language model (LLM) that handles both text encoding and language generation. This architecture demands significant memory and computational resources. For example, the FP16 model weights of a 13B-parameter VLM require approximately 26 GB of memory, excluding additional runtime memory for activations and key-value caching [3]. In contrast, many edge devices have limited memory and computational resources for VLM inference [2], [4].

Several model optimisation techniques, including quantisation, pruning, and knowledge distillation [2], [5], have been used to reduce model size and inference cost. However, these methods still have limitations. They can lower memory usage and improve deployment feasibility, but they may also require additional training and engineering effort [5], [4]. More importantly, even after compression, fully deploying a large VLM on a highly resource-constrained edge device may still be impractical [2], [3].

A distributed VLM provides a practical approach instead of full edge deployment of large VLMs [3]. In this architecture, the edge device deploys the early part of the model, such as the visual encoder and the projection module, and generates intermediate embeddings locally. These intermediate embeddings are then transmitted to a cloud or server-side component, which completes the remaining inference process. This architecture enables the system to reduce the computational cost on the edge device while still using a large VLM backend for complex vision-language reasoning tasks.

By transmitting intermediate embeddings instead of raw images, distributed VLMs can reduce the direct exposure of original input data. However, this architecture also introduces new security and privacy risks. The intermediate embeddings transmitted from the edge device to the cloud are often referred to as smashed data in the split learning literature [6]. Although they are not raw images, they may still contain sensitive information relating to the original input. Previous studies on split learning have shown that intermediate features can be vulnerable to reconstruction attacks [7], [8]. In such attacks, an adversary attempts to recover information about the original input from the transmitted intermediate representations.

Existing privacy-preserving approaches protect different stages of the machine-learning pipeline. However, many of them are designed for specific training tasks, retrieval tasks,

or computational models. Their suitability for generative split VLM inference remains unclear. In particular, there is limited system-level evaluation for protecting intermediate visual embeddings during edge-to-cloud transmission and cloud-side processing.

This study addresses this challenge by designing and evaluating a secure pipeline for handling intermediate embeddings in split VLM inference. The proposed system combines authenticated and encrypted transport, hardware-backed protected cloud execution, and remote attestation. The system is designed to protect intermediate embeddings during edge-to-cloud transmission and cloud-side processing without changing the existing VLM architecture. The evaluation focuses on end-to-end latency, process CPU time, throughput, and output consistency.

## II. Literature Review

This literature review is organised thematically rather than paper by paper because the project combines several connected research areas, including edge deployment of vision-language models, distributed VLMs, secure transmission, privacy-preserving machine learning, and system-level performance evaluation. Each theme is reviewed by considering its contribution to the secure handling of intermediate embeddings in split VLM inference, its main limitations, and its relevance to the proposed edge-cloud pipeline.

Table I summarises the main studies reviewed in this section.

### A. Deployment Challenges of Vision-Language Models

Vision-language models (VLMs) have become a powerful class of multimodal models as they can process visual and textual inputs. Modern VLMs typically combine a visual encoder, a projection model (cross-modal alignment model), and an LLM decoder. The high computational cost required for the deployment of VLMs at the edge is a significant challenge. Recent lightweight and edge-oriented VLM studies have attempted to reduce local deployment costs [2], [9], [4]. For instance, MobileVLM uses a downscaled LLaMA-based language model and an efficient projector to improve inference efficiency on resource-limited devices [9], while EdgeVL adapts large vision-language models to edge devices through knowledge distillation and quantisation-aware learning [4].

However, these approaches mainly focus on reducing local inference cost. Larger VLMs usually require stronger computational resources and are therefore more suitable for cloud-side deployment, but cloud-side deployment requires visual data to be transmitted from the edge to the cloud. Sending raw images can raise privacy problems because they may contain sensitive visual information, and frequent image transmission can create bandwidth and latency bottlenecks.

### B. Distributed VLMs and Intermediate Representations

A distributed VLM is a distributed deployment strategy in which a neural network is divided between an edge device and a cloud server at a selected split point. The edge device executes the early layers of the model and sends the generated intermediate representations to the cloud server for the remaining computation. This design can reduce the computational burden at the edge device, while the cloud-side can use a more capable model. Kang et al. [10] propose Neurosurgeon, a collaborative inference system that partitions DNN execution between a mobile device and a cloud server. Their work shows that selecting an appropriate split point can balance mobile-side computation, cloud-side computation, communication cost, and end-to-end latency. This idea has recently been extended from conventional DNNs to VLM deployment. Li et al. [3] propose a distributed VLM architecture in which the edge device runs a visual encoder and projection layer and sends projected visual embeddings to a central server, while the server embeds the text prompt, associates it with the corresponding visual embedding using a shared input ID, and concatenates the matched embeddings before LLM-based generation. In addition, the experiments show that the distributed VLM architecture improves token generation throughput by up to 33.54% compared with a centralised deployment, without relying on model compression techniques.

### C. Privacy Risks of Intermediate Embeddings

In distributed VLMs, intermediate embeddings must be transmitted from the edge device to the cloud server. Although transmitting embeddings may reduce privacy leakage risk more than directly uploading raw images, these embeddings are still derived from the original input and may therefore leak sensitive information about it.

Erdogan et al. [8] propose a set of attacks against split learning under an honest-but-curious server setting. Their work shows that a server can recover client input samples using only knowledge of the client-side model architecture and the intermediate representations received during split learning.

In a more recent study, Xu et al. [7] propose a Feature-Oriented Reconstruction Attack against split learning. The study shows that an honest-but-curious server can reconstruct the client's original input samples from the intermediate representations transmitted to the server. In the context of this project, this suggests that projected visual embeddings sent from the edge device to the cloud server may still contain information about the original visual input.

These studies demonstrate that intermediate embeddings in distributed VLM pipelines may be exposed to reconstruction attacks even under a passive honest-but-curious threat model. This highlights the need for a securing intermediate embeddings mechanism that protects intermediate visual embeddings during edge-to-cloud transfer in split VLM inference.

### D. Privacy-Preserving and Secure Inference Methods

Privacy protection for intermediate embeddings in distributed VLMs can be considered across different stages of the data lifecycle, including data in transit and data in use. During data transmission, authenticated encryption schemes, such as AES-GCM and ChaCha20-Poly1305, can protect the confidentiality and integrity of embeddings for split VLMs. In particular, ChaCha20-Poly1305 may provide

TABLE I
COMPARATIVE ANALYSIS OF THE REVIEWED LITERATURE

| Theme | Representative Papers | Main Findings and Limitations | Relevance to This Project |
|---|---|---|---|
| Edge Deployment of Vision–Language Models | VLM edge deployment survey; MobileVLM; EdgeVL [2], [9], [4] | Lightweight architectures, efficient projectors, knowledge distillation, and quantisation can reduce the computational and memory requirements of VLMs on resource-constrained devices. However, techniques such as knowledge distillation and quantisation-aware training may introduce additional training and engineering overhead. In addition, reducing model size may limit model capability, scalability, or performance on complex vision–language tasks. Therefore, fully deploying a capable VLM on highly constrained edge devices can remain challenging. | Motivates splitting the VLM pipeline between edge and cloud, while highlighting the need to protect data generated at the split point. |
| Distributed VLM Architectures | Neurosurgeon; distributed VLM architecture [10], [3] | Distributed VLMs partitions model execution between edge and cloud resources. Recent distributed VLM work sends projected visual embeddings from an edge-side visual encoder and projection module to a cloud-side language decoder. The split point affects communication cost, local computation, cloud computation, and end-to-end latency. | Provides the technical basis for the implemented pipeline, in which projected visual embeddings are generated at the edge and reconstructed for cloud-side VLM decoding. |
| Privacy Risks of Intermediate Representations | UnSplit; Feature-Oriented Reconstruction Attack [8], [7] | Intermediate representations (smashed data) can reveal information about client inputs under particular split-learning threat models. Honest-but-curious servers may use received representations and knowledge of the client-side model to reconstruct input-related information. | Supports the security motivation of the project: projected visual embeddings should not automatically be treated as non-sensitive data, even when raw images are not directly transmitted. |
| Privacy-Preserving and Secure Inference Methods | Federated Learning; FedAGRU; client selection; Homomorphic Encryption; Differential Privacy; privacy-preserving multimodal retrieval [11], [12], [13], [14], [15], [16] | FL mainly protects training data by keeping it on local clients, while HE enables encrypted computation at substantial computational cost. DP provides formal privacy guarantees through controlled noise addition, which may reduce utility. Combined multimodal designs can use feature obfuscation and encrypted storage, but may focus on retrieval or training rather than online VLM inference. | Shows that existing privacy-preserving approaches may not directly address the low-overhead protection of projected visual embeddings during split VLM inference. |
| Secure Transport and Protected Cloud Execution | ChaCha20-Poly1305; confidential computing; SecMdp; DarkneTZ [17], [18], [19], [20] | Authenticated encryption can protect embedding confidentiality and integrity during network transmission. Trusted execution environments can isolate selected data and model components during execution. However, protected execution introduces deployment, compatibility, and runtime-overhead considerations. | Directly informs the proposed system, which evaluates protected embedding transmission and hardware-backed protection for cloud-side processing. |

efficient encryption performance on devices without AES hardware acceleration [17]. However, conventional encryption can protect data during transmission or storage. Once the cloud server decrypts the intermediate embeddings for inference, the plaintext embeddings may be exposed in cloud-side memory. Therefore, conventional encryption alone cannot provide data-in-use protection.

Trusted execution environments (TEEs) protect sensitive data while it is being processed. A TEE provides an isolated execution environment and protected memory for sensitive code and data. This may protect decrypted intermediate embeddings and selected inference components on the cloud side [18]. In contrast, transport encryption protects embeddings only while they travel between the edge and cloud.

Bai et al. proposed SecMdp, an SGX-based framework for secure multimodal learning. It uses MM PathORAM to load data securely from untrusted storage while reducing access-pattern leakage during data loading [19]. Mo et al. proposed DarkneTZ, which uses ARM TrustZone to protect sensitive DNN layers during execution. Sensitive layers run inside the TEE, while the remaining layers run in the normal operating system [20]. These studies show that TEEs can protect sensitive data and model components during DNN inference. This capability may also be applied to protect decrypted intermediate embeddings during cloud-side processing.

In addition to hardware-backed isolation, cryptographic approaches can protect data during computation. Homomorphic encryption (HE) provides an approach that allows computation over encrypted data. However, encrypted neural network inference usually introduces significant computational overhead and often requires specialised model or operator design. For instance, Rho et al. [14] proposed an encryption-friendly LLM architecture that uses LoRA fine-tuning and replaces the softmax operation with Gaussian kernels to reduce the cost of encrypted transformer computation. Although this study demonstrates the potential of HE-based private inference, it also shows that applying encrypted computation to transformer models often requires architecture-level adaptation and the replacement of computationally expensive operations, such as softmax, with encryption-friendly alternatives that can be evaluated efficiently under HE. However, the evaluation is limited to a relatively small transformer model similar to BERT, rather than a full-scale decoder-only LLM or VLM.

Differential privacy provides a formal privacy framework

by adding calibrated noise according to a privacy budget [15]. However, stronger privacy guarantees may reduce model utility. This makes DP less directly suitable for split VLM scenarios.

Federated learning provides another privacy-preserving approach by keeping training data on local clients and sharing model updates instead of raw data. This can reduce direct exposure of user data to the cloud [11]. However, FL requires repeated local training and communication rounds, which can introduce considerable computation and energy overhead on resource-constrained edge devices. Some studies attempt to reduce these costs. Chen et al. [12] propose FedAGRU, which uses attention-based aggregation to accelerate convergence and reduce the number of communication rounds. Bhavani et al. [13] use client selection to reduce communication and computational overhead by selecting more secure and resource-rich devices for FL. However, these methods mainly optimise the training stage of FL. They do not directly address the inference-stage problem in cloud-based VLM deployment, in which raw images or embeddings still need to be transmitted from the edge device to the cloud server.

### *E. Research Gap*

Recent studies have explored both individual and combined privacy-preserving approaches for multimodal machine learning systems. These approaches protect different types of information at different stages of the processing pipeline. For example, FL keeps raw training data on client devices. Clients usually send model parameters or model updates to a central server instead of sharing the raw data [11]. This reduces the direct exposure of sensitive training data. Fully homomorphic encryption allows a server to perform specific computations directly on encrypted data [14]. The data remain encrypted during network transmission and cloud-side computation. This provides strong data confidentiality during both transmission and computation.

Beyond these individual approaches, some studies further combine multiple mechanisms to address the privacy requirements of specific applications. For example, PMIRS combines federated learning, block-wise embedding obfuscation, AES encryption, and Diffie–Hellman-based key management [16]. These mechanisms protect both the training and inference stages of multi-user image–text retrieval. Another combined approach is SpFedRec, a split-federated recommendation system [21]. It combines split learning, federated learning, and secret sharing. This design reduces the exposure of local user data. It also protects intermediate information exchanged during distributed training and recommendation.

However, these approaches are generally designed around specific tasks, processing stages, or computational models. The embedding-obfuscation mechanism in PMIRS is primarily designed for multimodal retrieval using cosine similarity. The study does not show that the transformed embeddings can be used directly by an existing autoregressive VLM decoder for text generation. Therefore, applying this transformation to a generative VLM may require recovering the original embeddings before decoding or adapting the multimodal projector or decoder. The reviewed split-federated recommendation system reduces the direct exposure of local user data [21]. The reviewed system does not explicitly evaluate the protection of intermediate representations or inference state during cloud-side processing.

FHE-based approaches have a different limitation. FHE-friendly designs can reduce inference cost through operator replacement and LoRA-based model adaptation [14]. These designs still require changes to a standard LLM and introduce substantial computational overhead compared with plaintext inference.

Therefore, existing studies have not fully evaluated a security framework for generative split VLMs that works with existing models. The framework should protect intermediate embeddings during edge-to-cloud transmission. It should also protect decrypted embeddings and inference state during cloud-side processing. It should achieve this without changing the embedding representation, retraining the cloud-side decoder, or replacing existing Transformer operators. The edge client should also verify the protected cloud environment before sending sensitive embeddings.

This project addresses this gap through a secure split VLM inference pipeline. It combines authenticated and encrypted transport, TEE-based cloud execution, and remote attestation. The study also measures the system-level overhead introduced by these mechanisms.

## III. Methodology

### *A. Research Design Overview*

This study presents a secure split vision-language model (VLM) inference framework for edge-cloud deployment. The framework divides the VLM inference process between an edge device and a cloud server. On the edge side, the framework performs visual encoding and projection and transmits the generated intermediate visual embeddings to the cloud server through an authenticated and encrypted channel. On the cloud side, it performs text prompt tokenisation and embedding, followed by LLM decoding. This study does not aim to develop a new split VLM architecture or a new cryptographic algorithm. Instead, it evaluates the security and performance implications of applying secure transport, remote attestation, and hardware-backed protected execution to an existing split VLM inference pipeline. The study aims to identify a suitable balance between security protection and system-level efficiency. The final evaluation is grounded in reproducible inference outputs and system-level measurements, including end-to-end latency, process CPU time, throughput, and output consistency.

### *B. Threat Model and Security Requirements*

**Attacker model.**

This study considers two main types of attackers. The first is an active network attacker. This attacker can observe, modify, delay, replay, drop, or inject packets between the edge device and the cloud server. The attacker may also attempt a man-in-the-middle attack or impersonate a communication endpoint.

The second is a privileged cloud-side attacker. This attacker may control the host operating system, hypervisor, or other cloud infrastructure. The attacker may attempt to inspect process memory, attach a debugger, modify runtime data, or inject code into the cloud-side inference process. The main target is the decrypted visual embeddings and inference state used during cloud-side processing.

**Trusted components.**

The edge client is trusted. The edge-side visual encoder, projection module, and cryptographic implementation are also trusted. The Intel TDX hardware and attestation root are assumed to operate correctly. The cloud-side VLM application is assumed to be trusted. The implemented remote attestation procedure verifies that received evidence originates from a valid Intel TDX environment and is associated with the client-provided nonce.

**Untrusted components.**

The public network, host operating system, hypervisor, and privileged cloud infrastructure are not trusted. The edge client must verify the protected cloud environment before sending intermediate embeddings.

**Security requirements.**

The security goals are to protect the confidentiality and integrity of intermediate embeddings during network transmission and cloud-side processing. The system must also allow the edge client to verify the remote execution environment before releasing sensitive embeddings. If remote attestation fails, the client must not send the embedding payload.

**Out-of-scope threats.**

This study does not consider a compromised edge client, physical attacks, denial-of-service attacks, or malicious code already running inside the TD. Misuse by the authorised VLM application is also outside the scope. The evaluation does not attempt to address all TEE side-channel attacks.

### *C. System Architecture and Pipeline Overview*

Fig. 1 shows the overall inference pipeline and the placement of the proposed security mechanisms. First, the edge device generates the intermediate visual embeddings. Next, the system establishes a TLS session and performs remote attestation. Upon successful verification, the embeddings are transmitted to the protected cloud environment. Finally, the cloud-side VLM processes the embeddings and generates the text output.

For each inference request, the input image is first processed by the visual encoder on the edge device. The resulting visual features are transformed by the projection module into intermediate visual embeddings that are compatible with the input space of the cloud-side language model. Upon successful remote attestation, the intermediate embeddings are serialised and transmitted through the established TLS channel.

Before transmitting the embedding payload, the edge client first establishes a TLS session with the cloud server. It then uses remote attestation to verify the protected cloud execution environment.

On the cloud side, the embedding payload is first received and reconstructed inside the Intel TDX-protected execution environment. Next, the reconstructed visual embeddings are combined with the tokenised text prompt to construct the multimodal input. The LLM decoder then processes the multimodal representation and generates the final textual output. Overall, TLS protects the embedding payload during transmission, while Intel TDX protects the embeddings and inference state during cloud-side processing.

### *D. Split VLM Inference and Embedding Transmission*

In this study, the split point is placed after the projection module in the implemented split VLM pipeline. The edge device serialises the projected visual embeddings into a byte payload and sends it to the cloud server. To enable reconstruction on the cloud server, the transmitted payload includes metadata describing the embedding tensor shape and data type. The cloud server deserialises the received payload and reconstructs the visual embedding tensor.

### *E. Transport Security with TLS 1.3*

TLS 1.3 is used to protect the transmission of embeddings between the edge client and the cloud server. Before transmitting any embedding information, the edge client establishes a TLS session with the cloud server and validates the server certificate. This process authenticates the communication endpoint and establishes an encrypted channel.

The TLS 1.3 channel protects the confidentiality and integrity of the embedding metadata and payload during network transmission. It prevents an untrusted network observer from reading the transmitted data and from modifying it without detection.

### *F. Protected Cloud Execution with Intel TDX*

Intel TDX protects intermediate embeddings during cloud-side processing. It creates a hardware-isolated environment called a Trust Domain (TD). The cloud-side VLM runs inside a confidential virtual machine in the TD. The TD protects its memory and execution state from the untrusted host operating system and hypervisor.

After the embedding payload reaches the TD, the cloud-side application processes it inside the protected environment. Embedding reconstruction, multimodal input construction, and language generation all take place inside the TD.

Intel TDX protects the intermediate embeddings and inference state from the untrusted host operating system and hypervisor. An attacker who compromises the host cannot directly read or extract the protected embeddings or inference state.

### *G. Remote Attestation and Trust Establishment*

Remote attestation allows the edge client to verify the protected cloud environment before sending sensitive embeddings. The Intel TDX environment generates a quote as hardware-backed attestation evidence.

The edge client first validates the TLS server certificate to authenticate the cloud server endpoint. It then requests and verifies the TDX quote through the same authenticated

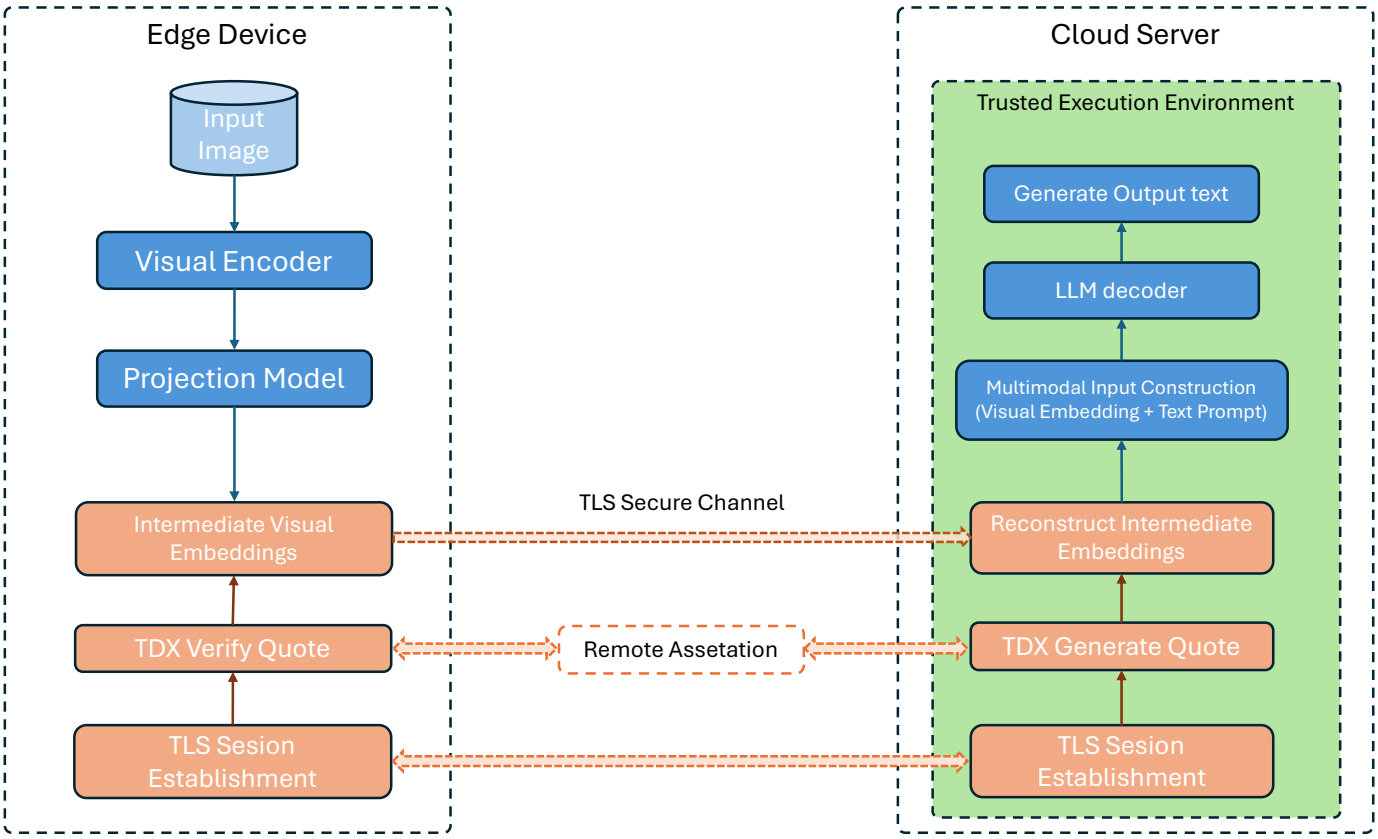



Fig. 1. System Architecture of the Proposed Secure Split VLM Inference Framework

TLS session. Quote verification confirms that the attestation evidence was generated by a valid Intel TDX environment and was associated with the client-provided nonce. In the implemented deployment, the TLS endpoint and the attestation service run within the same TDX VM. Together, certificate validation and quote verification provide assurance that the authenticated cloud endpoint operates within a TDX-protected environment. Embedding transmission proceeds only after both verification steps are completed successfully. Otherwise, the transmission is aborted.

### *H. Experimental Setup*

*1) Dataset:* This study uses the AI2 Diagrams (AI2D) subset from HuggingFaceM4/the_cauldron as the evaluation dataset. HuggingFaceM4/the_cauldron is a multimodal dataset released by the HuggingFaceM4 team at Hugging Face. The AI2D subset was introduced in A Diagram Is Worth A Dozen Images. It contains science diagrams, each paired with a multiple-choice question and a corresponding answer [22]. The AI2D subset included in HuggingFaceM4/the_cauldron contains approximately 2,400 samples, which makes it suitable for rapid evaluation in this study.

This dataset is suitable for this study because it provides both images and associated prompts. The proposed evaluation requires the same image-question pairs to be processed by both the original nanoVLM inference pipeline and the proposed secure split-inference pipeline. By using the same inputs, the study can compare whether the secure split-inference pipeline produces inference outputs that are consistent with those of the original nanoVLM pipeline.

In this study, the dataset has two main roles in the evaluation. First, it is used to measure the performance overhead of the proposed secure split-inference framework, including process CPU time and latency. Second, it is used to check whether the secure split-inference pipeline produces outputs consistent with those of the original nanoVLM inference. The VQA-style multiple-choice format allows the model outputs to be constrained to simple answer options, such as A, B, C, or D. This makes the output comparison more controlled and reliable, because the results can be compared directly as text strings.

This is important for this study, as the purpose of the evaluation is not to assess the general accuracy of nanoVLM, but to determine whether the proposed secure transmission mechanisms maintain consistent inference outputs in the split-inference pipeline. To ensure a controlled comparison, output consistency in this study mainly depends on using the same inputs, the same model weights, a fixed random seed, and greedy decoding.

Other datasets, such as ImageNet and selected COCO-sample datasets, were also considered. However, these datasets mainly provide image labels without prompts. Therefore, they are not suitable for the prompt-based consistency evaluation required in this study.

*2) Model configuration:* This study uses the pre-trained `lusxvr/nanoVLM-230M-8k` checkpoint as the evaluation

model. It contains a SigLIP2-base-patch16-512 visual encoder, a projection module, and a SmolLM2-360M-Instruct language decoder. The split point is placed after the projection module. The visual encoder and projection module run on the edge device. The language decoder runs on the cloud server.

### I. Evaluation Configurations and Baselines

This evaluation compares three configurations with different protection scopes. First, the baseline configuration uses HTTP over TCP without additional transport protection. Second, the secure transport configuration uses HTTP over TLS 1.3 over TCP to provide encrypted and authenticated embedding transmission. Third, the enhanced protection configuration uses HTTP over TLS 1.3 over TCP together with a hardware-backed protected cloud execution environment. In addition, the original nanoVLM inference pipeline is used as a reference for output consistency validation. The outputs of the original nanoVLM pipeline are compared with those of the split-inference configurations. This comparison determines whether the split and protection mechanisms preserve inference results under the same inputs and generation settings.

### J. Experimental Environment Validation

Before comparing performance results, the experimental environments were checked. This was necessary because the normal VM and the TDX VM were separate virtual machines. The goal was to reduce the effect of hardware and software differences on the results. Table II shows that the two environments used the same operating system, kernel version, CPU architecture, vCPU count, software stack, and package environment. Therefore, the observed performance differences were mainly associated with Intel TDX rather than differences in the VM configurations.

### K. Evaluation Metrics and Measurement Procedure

The performance evaluation measures the system-level overhead introduced by secure transport and protected cloud execution in the split VLM pipeline. The collected metrics include end-to-end latency, process CPU time, and throughput. Output consistency is also measured to determine whether the different configurations produce the same inference results.

Wall-clock latency and process CPU time measure different aspects of system performance. Wall-clock latency includes connection setup, network waiting, and response time. Process CPU time measures only the CPU time consumed by the process. It excludes time spent waiting for network communication or CPU scheduling. Therefore, the two values are not expected to be equal.

All configurations are evaluated by repeatedly executing the same workload under the same experimental conditions. The same model weights, input samples, prompts, and generation settings are used in each configuration. The results are compared with the baseline split-inference configuration to identify the additional overhead introduced by TLS 1.3 and Intel TDX.

## IV. Results & Analysis

### A. Functional and Output Consistency Validation

The functional validation confirmed that the complete split-inference pipeline operated correctly. For each input, the edge device processed the image and generated the projected visual embeddings. The embeddings were then transmitted to the cloud server. The cloud application reconstructed the embedding tensor, combined it with the text prompt, and generated the final text response. This result showed that the model could be partitioned between the edge and cloud while preserving the inference workflow.

Output consistency was evaluated by comparing the original nanoVLM pipeline with the split-inference configurations. All configurations used the same image–question pairs, model weights, prompts, tokenizer, and generation settings. Greedy decoding was used to minimise stochastic variation during text generation.

A total of 1000 samples were evaluated. The generated responses from the split-inference configurations matched the original nanoVLM outputs for all samples, giving an exact output consistency rate of 100%. This result indicates that model partitioning, embedding serialisation, network transmission, TLS protection, and execution in the TDX-enabled cloud environment did not alter the generated answers in the evaluated workload.

### B. Security Mechanism Validation and Threat-Model Analysis

This section evaluates the security protection of each system configuration. The analysis follows the threat model defined earlier. Three configurations are compared: plain HTTP split inference, TLS-protected split inference, and TLS with Intel TDX protected split inference.

In the plain HTTP configuration, the raw image stays on the edge device. This reduces direct exposure of the original image to the cloud server. However, the projected visual embeddings are sent without transport-layer protection. A network attacker could observe or modify these embeddings during transmission. The cloud runtime is also not protected from a privileged cloud operator or a compromised host operating system.

In the TLS-protected configuration, TLS 1.3 protects the embedding transmission. It provides confidentiality and integrity for the embeddings during transfer. This protects the system from passive eavesdropping and active network modification. However, TLS only protects data in transit. Once the embeddings arrive at the cloud server, they are processed in a normal cloud runtime. Therefore, this configuration does not protect the reconstructed embeddings or inference state from privileged cloud-side software.

In the TLS and Intel TDX configuration, TLS protects the embeddings during transmission. Intel TDX protects the cloud-side execution environment. The cloud application runs inside a TDX-protected confidential virtual machine. This reduces the exposure of reconstructed embeddings, model inputs, and intermediate inference state to the host operating system and hypervisor. Remote attestation is performed before split inference starts. This allows the edge device to validate

TABLE II
EXPERIMENTAL ENVIRONMENT VALIDATION

| Item | Normal VM | TDX VM | Conclusion |
|---|---|---|---|
| OS | Ubuntu 26.04 LTS | Ubuntu 26.04 LTS | Same |
| Kernel | 7.0.0-1005-gcp | 7.0.0-1005-gcp | Same |
| Architecture | x86_64 | x86_64 | Same |
| vCPU count | 4 | 4 | Same |
| CPU family/model | family 6, model 143 | family 6, model 143 | Same CPU generation |
| Topology | 1 socket, 2 cores/socket, 2 threads/core | Same | Same |
| Hypervisor | KVM / Google | KVM / Google | Same cloud virtualization layer |
| Python | Python 3.14.4 | Python 3.14.4 | Same |
| pip version | pip 25.1.1 | pip 25.1.1 | Same |
| pip packages | 85 packages | 85 packages | Same package list |
| OpenSSL | OpenSSL 3.5.5 | OpenSSL 3.5.5 | Same |
| GCC | GCC 15.2.0 | GCC 15.2.0 | Same |

attestation evidence associated with the client-provided nonce before sending embeddings.

Overall, the three configurations provide increasing levels of protection. HTTP-based split inference keeps the raw image on the edge device, but does not protect embeddings in transit. TLS adds protection for embeddings during transmission. TLS combined with Intel TDX further protects cloud-side processing. This layered design matches the threat model because it protects sensitive visual data and intermediate representations from both network attackers and privileged cloud-side adversaries.

### C. End-to-End Performance Results

This section reports the end-to-end performance of the evaluated split-inference configurations. It also examines the additional cost of TLS protection and Intel TDX. The comparison focuses on latency, throughput, and process CPU time.

The total split-inference latency consists of three main parts. First, the edge device processes the image and generates the projected visual embeddings. Second, the embeddings are serialised and sent to the cloud server. Third, the cloud server reconstructs the embedding tensor and generates the final text response. In the TLS configurations, the transmission stage also includes the cost of encrypted communication and authentication. In the TDX configuration, the cloud-side inference stage runs inside the TDX virtual machine.

Table III summarises the measured latency results for the evaluated configurations.

TABLE III
LATENCY RESULTS

| Config. | VM | Embedding (s) | Network (ms) | Cloud (s) |
|---|---|---|---|---|
| HTTP | Normal VM | 18.9587 | 4.0274 | 14.8161 |
| HTTP | TDX VM | 18.9045 | 4.4494 | 16.4645 |
| AES-GCM | Normal VM | 18.7869 | 20.1833 | 14.8943 |
| AES-GCM | TDX VM | 18.8247 | 20.6957 | 16.5837 |
| ChaCha20 | Normal VM | 18.7692 | 20.2658 | 14.9650 |
| ChaCha20 | TDX VM | 18.7151 | 20.6962 | 16.6273 |

The edge embedding latency was stable across all tested configurations. It was approximately 18.7–19.0 seconds. This result is expected, because the edge-side computation did not change. The network overhead was much smaller than the model computation time. HTTP had about 4 ms of network overhead. The HTTPS configurations increased this value to about 20 ms when a newly established connection was used. The effect of connection reuse is evaluated separately in Table V. However, the network overhead was still small compared with the edge embedding time and the cloud inference time.

Table IV shows the throughput results. Throughput showed a similar pattern to the latency results. The normal VM processed about 1.77 images per minute. The TDX VM processed about 1.69 images per minute. This means that throughput decreased by approximately 4.5–4.9%. This reduction is consistent with the higher cloud-side inference time in the TDX VM.

TABLE IV
THROUGHPUT RESULTS

| Configuration | VM | Throughput (images/min) | TDX reduction |
|---|---|---|---|
| HTTP | Normal VM | 1.7700 | – |
| HTTP | TDX VM | 1.6905 | 4.48% |
| HTTPS + AES-GCM | Normal VM | 1.7741 | – |
| HTTPS + AES-GCM | TDX VM | 1.6879 | 4.86% |
| HTTPS + ChaCha20 | Normal VM | 1.7743 | – |
| HTTPS + ChaCha20 | TDX VM | 1.6885 | 4.84% |

Overall, the end-to-end results show that model computation was the main cost in the split-inference pipeline. Network transmission was not the main bottleneck. The HTTPS configurations showed higher transmission latency than HTTP, but the increase remained small relative to the edge and cloud computation time. This measurement represents the combined cost of TLS-protected communication and does not separately quantify the costs of authentication and data encryption. Intel TDX introduced a larger but still moderate performance overhead. This overhead mainly appeared during cloud-side inference.

### D. TLS Performance Overhead

This subsection evaluates the additional performance overhead introduced by TLS 1.3 compared with the unprotected HTTP baseline. The analysis separates the estimated TLS handshake contribution from the steady-state protected data transfer overhead.

TLS 1.3 introduces two types of overhead: connection establishment and protected data transfer. During connection establishment, the TLS handshake authenticates the server, negotiates the security parameters, and derives the session keys. During data transfer, TLS encrypts and authenticates the embedding data during transmission.

To separate these costs, the experiment tested HTTPS requests with newly established and reused connections. A request over a newly established connection included TCP connection setup, the TLS handshake, and protected data transfer. A request over a reused TLS connection did not require a new TCP connection or a new TLS handshake. The latency difference between the two requests represented the combined TCP and TLS connection-establishment cost. The corresponding HTTP connection-establishment cost was then subtracted to estimate the TLS handshake contribution.

In the implemented workflow, the TDX attestation request and the subsequent embedding request reused the same TLS connection. Therefore, the embedding transfer did not require a new TCP connection or TLS handshake.

Table V compares the request latency over newly established and reused connections.

TABLE V
REQUEST LATENCY OVER NEWLY ESTABLISHED AND REUSED CONNECTIONS

| VM | Transport | New connection latency (ms) | Reused connection latency (ms) |
|---|---|---|---|
| Normal VM | HTTP | 4.027 | 3.106 |
| Normal VM | TLS/AES-GCM | 20.183 | 3.310 |
| Normal VM | TLS/ChaCha20 | 20.266 | 3.279 |
| TDX VM | HTTP | 4.449 | 3.228 |
| TDX VM | TLS/AES-GCM | 20.696 | 3.518 |
| TDX VM | TLS/ChaCha20 | 20.696 | 3.548 |

A request over a newly established HTTPS connection took approximately 20.18–20.70 ms. After connection reuse, the request latency decreased to approximately 3.28–3.55 ms. The steady-state TLS overhead was calculated by subtracting the reused HTTP latency from the corresponding reused HTTPS latency. It ranged from 0.173 to 0.320 ms per request. After correcting for the HTTP connection-establishment cost, the estimated TLS handshake contribution ranged from 15.93 to 16.07 ms. These results show that most of the initial HTTPS latency came from connection establishment. After connection reuse, TLS introduced only a small overhead compared with the edge and cloud computation times. No substantial difference was observed between AES-GCM and ChaCha20 under the evaluated settings.

### *E. Intel TDX Performance Overhead*

This subsection examines the performance overhead introduced by Intel TDX during cloud-side inference. The normal VM and the TDX VM were compared under the same three transport configurations.

As shown in Table III, the TDX VM showed higher cloud inference latency in all three transport configurations. Compared with the normal VM, the average increase was approximately 11.19%.

Table VI shows a similar pattern in cloud process CPU time. The cloud CPU-time increases were 11.13%, 11.33%, and 11.07% for the three transport configurations, respectively. The average increase was approximately 11.18%. In contrast, the edge process CPU time remained stable because the edge-side computation was not affected by the cloud VM type.

The similar increases in cloud inference latency and cloud process CPU time indicate that the observed overhead was mainly associated with cloud-side execution in the TDX VM. The increase was also similar across the three transport configurations. Therefore, the TDX overhead was not strongly affected by the selected transport protocol or TLS cipher suite.

However, these results only locate the overhead in cloud-side execution. They do not identify which stage of cloud inference contributed most to the increase. Therefore, the cloud generation process was further profiled.

### *F. Autoregressive Decoder Profiling*

To further locate the source of the TDX overhead, the cloud-side generation process was divided into prompt prefill, autoregressive decoding, and post-processing. Prompt prefill processes the input prompt and visual embeddings before token generation begins. Autoregressive decoding then generates the output one token at a time. Finally, output post-processing converts the generated token IDs into text, removes special tokens, and prepares the final response.

As shown in Table VII, the total generation latency increased from 14.938 seconds on the normal VM to 16.529 seconds on the TDX VM. This was an increase of 1.591 seconds, or approximately 10.65%.

The prefill latency increased from 1.272 seconds to 1.334 seconds. The absolute increase was 0.062 seconds. In contrast, autoregressive decoding latency increased from 13.665 seconds to 15.195 seconds. This was an increase of 1.530 seconds.

The decoding increase accounted for approximately 96.2% of the total generation-latency difference:

$$\frac{15.195 - 13.665}{16.529 - 14.938} \times 100\% = 96.2\%. \tag{1}$$

Post-processing latency increased by only 0.027 milliseconds. This increase made only a small contribution to the total latency. These results show that autoregressive decoding was the main source of the additional TDX generation latency.

*1) EOS-Based Early Stopping:* Runtime logs and code inspection showed that the original implementation continued autoregressive decoding until it reached the configured limit of 300 tokens, even after an EOS token had been generated. Tokens generated after EOS were removed during post-processing. But their decoding iterations had already consumed computational resources. Therefore, EOS early stopping was introduced to stop decoding after an EOS token was generated and avoid unnecessary computation.

Table VIII compares generation latency before and after EOS-based early stopping. The optimisation reduced generation latency by approximately 67.2–67.7% across all configurations. This reduction resulted from removing unnecessary decoding iterations after the EOS token.

TABLE VI
CPU-TIME RESULTS FOR THE NORMAL VM AND THE INTEL TDX VM

| Configuration | VM | Edge CPU time (s) | Transfer CPU time (ms) | Cloud CPU time (s) | CPU time increase |
|---|---|---|---|---|---|
| HTTP | Normal VM | 37.997 | 3.00 | 29.628 | – |
| HTTP | TDX VM | 37.871 | 3.00 | 32.926 | 11.13% |
| HTTPS + AES-GCM | Normal VM | 37.648 | 12.00 | 29.787 | – |
| HTTPS + AES-GCM | TDX VM | 37.724 | 10.00 | 33.163 | 11.33% |
| HTTPS + ChaCha20 | Normal VM | 37.507 | 11.00 | 29.932 | – |
| HTTPS + ChaCha20 | TDX VM | 37.597 | 11.00 | 33.246 | 11.07% |

TABLE VII
BREAKDOWN OF CLOUD-SIDE GENERATION LATENCY

| VM | Prefill (s) | Decode (s) | Post. (ms) | Total (s) |
|---|---|---|---|---|
| Normal VM | 1.272 | 13.665 | 0.434 | 14.938 |
| TDX VM | 1.334 | 15.195 | 0.461 | 16.529 |

However, the relative TDX overhead decreased only slightly. It remained between 8.73% and 9.66% after EOS early stopping. This result shows that early stopping reduced unnecessary computation on both VMs, but it did not remove the per-step overhead of TDX execution.

*2) Decoder Operation-Level Profiling:* Although EOS early stopping reduced the total generation latency, the relative TDX overhead remained close to 9%. This result indicates that the remaining decoding iterations were still slower on the TDX VM. Therefore, the decoder was further profiled at the operation level.

The decoder forward pass includes normalisation, query-key-value (QKV) projection, rotary positional embedding (RoPE) operations, key-value (KV) cache operations, attention, output projection, and the multilayer perceptron (MLP). The mean additional TDX time for each measured operation was calculated as follows:

$$\Delta T_{\text{operation}} = \overline{T}_{\text{operation,TDX}} - \overline{T}_{\text{operation,Normal}}. \tag{2}$$

As shown in Table IX, the TDX overhead was distributed across several decoder operations. The MLP showed the largest increase, adding an average of 146 ms per request.

The multilayer perceptron (MLP) is the feed-forward component of each Transformer block. It applies separate gate and up projections to the input. The gate projection uses the SiLU activation function, and its output is multiplied element by element with the up-projection output. The down projection then maps the result back to the embedding dimension. These three dense projections are repeated in every decoder layer and decoding step. Their computation and memory costs may explain why the MLP showed the largest absolute TDX time increase.

### G. Threats to Validity

The experiments were conducted on Google Cloud Platform (GCP) virtual machines. The underlying cloud infrastructure and host workload may vary between experimental runs. These variations may affect the measured inference latency and CPU time. To reduce their impact, each configuration was tested three times, and the mean values were reported.

The normal VM and the TDX VM used the same CPU model, number of vCPUs, software environment, model, and inference settings. These controls reduced performance differences caused by system configuration. Process CPU time was also measured together with wall-clock latency. This metric measures the time spent executing the program on the CPU and excludes time spent waiting for CPU scheduling.

The evaluation was conducted using nanoVLM, the AI2D subset, and a GCP environment. Therefore, the measured performance results may not directly generalise to other models, workloads, hardware configurations, or confidential computing platforms.

## V. CONCLUSION AND FUTURE WORK

### A. Summary of Findings

This project designed and evaluated a secure split-inference framework for a vision-language model. The edge device processed the input images and generated projected visual embeddings. The projected visual embeddings were transferred to the cloud, while the raw image remained on the edge device. The cloud server reconstructed the embedding tensor and generated the text response.

The proposed security scheme protected data in two states: data in transit and data in use. During transmission, TLS 1.3 provided authentication, confidentiality, and integrity protection for the visual embeddings. During cloud-side processing, Intel TDX protected the reconstructed embeddings and inference state. Remote attestation was performed before the embeddings were transferred. It validated attestation evidence associated with the client-provided nonce.

The performance results showed that model computation was the main cost in the pipeline. HTTPS increased the transfer latency compared with HTTP. The measured increase was small relative to the edge-side and cloud-side computation time. Intel TDX introduced a larger overhead during cloud-side inference. Cloud inference latency and cloud process CPU time increased by approximately 11% on the TDX VM. Similar increases were observed across HTTP, AES-GCM, and ChaCha20 configurations.

Further profiling showed that autoregressive decoding was the main source of the additional TDX generation latency. EOS early stopping removed unnecessary decoding iterations and reduced generation latency by approximately 67%, while the relative TDX overhead remained close to 9%. Operation-level profiling showed that the multilayer perceptron (MLP) had the largest measured absolute increase among the decoder operations.

TABLE VIII
GENERATION LATENCY AND RELATIVE TDX OVERHEAD BEFORE AND AFTER EOS EARLY STOPPING

| Transport | Normal before (s) | TDX before (s) | Normal after (s) | TDX after (s) | TDX overhead before | TDX overhead after |
|---|---|---|---|---|---|---|
| HTTP | 14.938 | 16.529 | 4.863 | 5.333 | 10.66% | 9.66% |
| AES-GCM | 14.931 | 16.322 | 4.901 | 5.344 | 9.32% | 9.02% |
| ChaCha20 | 14.965 | 16.364 | 4.855 | 5.280 | 9.35% | 8.73% |

TABLE IX
MEAN ADDITIONAL TDX TIME FOR MEASURED DECODER OPERATIONS

| Decoder operation | Additional TDX time (ms) |
|---|---|
| MLP | 146.00 |
| Normalisation | 50.00 |
| QKV projection | 46.00 |
| Output projection | 42.00 |
| KV repeat | 29.00 |
| Attention core | 28.00 |
| Rotary operation | 14.00 |
| KV concatenation | 12.00 |
| RoPE value generation | 0.03 |

Overall, the results show that TLS 1.3 and Intel TDX can be combined with split VLM inference to protect intermediate embeddings during transmission and cloud-side processing. TLS added a limited overhead when data was transferred over an established connection. Intel TDX added a moderate overhead during cloud-side inference. For the evaluated workload, the framework provided additional protection with a moderate and measurable performance cost.

### *B. Limitations and Future Work*

This study has several limitations. It evaluates only one VLM checkpoint, one dataset, one cloud provider, and one Intel TDX configuration. The security evaluation is based on threat-model analysis and functional validation. It does not include direct attack experiments. The implementation does not compare MRTD or RTMR values with an approved workload policy. It also does not bind the TLS public key to the attestation evidence. In addition, changes in the underlying cloud infrastructure may affect the performance results.

Future work should test the framework with more VLMs and datasets. Different input sizes and generation workloads should also be evaluated. The experiments could be extended to other cloud platforms, hardware configurations, and confidential-computing technologies. These could include AMD- and Arm-based systems. Such evaluations would show whether the security design and performance results can be generalised beyond the evaluated Intel TDX environment.

Remote attestation could be more closely integrated with TLS endpoint authentication. For example, the TLS public key or certificate could be linked to the attested TDX environment. The client could then verify the server identity and the trusted execution environment through one authentication process.

Further performance optimisation should focus on autoregressive decoding and MLP operations. Possible methods include optimised inference kernels, model quantisation, batching, and more efficient cache management. Hardware performance counters could provide more information about the remaining TDX overhead. Baseline measurements for each decoder operation could also help identify its source.

## VI. USE OF AI

AI-assisted tools were used in a limited and supportive capacity during this study. Their use was limited to improving understanding and writing quality. AI tools did not replace the author's own work in the design, implementation, evaluation, or analysis of the study.

AI tools were applied in several areas of this study. For literature review, large language models (LLMs) such as ChatGPT and Claude were used to explain technical concepts and help understand the main ideas of research papers. Google Scholar Labs were used to discover relevant literature through search and recommendation features.

For writing support, AI-assisted tools such as Grammarly were used for grammar correction, spelling checks, sentence clarity, and general readability improvements. AI tools were also used to help organise notes and structure parts of the Overleaf document.

For implementation and experimental evaluation, AI tools were used to assist with debugging, interpreting error messages, and explaining the implementation details of open-source frameworks, libraries, and performance measurement tools, including nanoVLM, the Hugging Face Datasets library and API, PyTorch, benchmarking and profiling libraries. AI tools were also used to support the understanding of Python, Bash shell scripts and system integration issues.

The use of AI was limited to assistance and guidance areas. The study topic, research objectives, methodology design, implementation choices, experimental design, and evaluation procedures were determined and reviewed by the author. All code implementations, modifications, experimental data, and interpretations of results were checked and validated by the author.

AI tools were not used to generate experimental results, fabricate references, or create datasets. All references included in this dissertation were selected and reviewed by the author based on the literature examined in this study.

## ACKNOWLEDGMENT

The author thanks Dr Joseph Lemley for his supervision, guidance, and feedback throughout this capstone project.

## REFERENCES

[1] D. Zhang, Y. Yu, J. Dong, C. Li, D. Su, C. Chu, and D. Yu, "Mm-llms: Recent advances in multimodal large language models," *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 12 401–12 430, 2024.

[2] A. Sharshar, L. U. Khan, W. Ullah, and M. Guizani, "Vision-language models for edge networks: A comprehensive survey," *IEEE Internet of Things Journal*, 2025.

[3] Y. Li, D. Gumaste, M. K. Turkcan, J. Ghaderi, G. Zussman, and Z. Kostic, "Distributed vlms: Efficient vision-language processing through cloud-edge collaboration," in *2025 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*. IEEE Computer Society, 2025, pp. 280–286.

[4] K. Cai, Z. Duan, G. Liu, C. Fleming, and C. X. Lu, "Self-adapting large visual-language models to edge devices across visual modalities," in *European Conference on Computer Vision*. Springer, 2024, pp. 301–318.

[5] T. Wang, W. Zhou, Y. Zeng, and X. Zhang, "Efficientvlm: Fast and accurate vision-language models via knowledge distillation and modal-adaptive pruning," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 13 899–13 913.

[6] N. D. Pham and N. Chilamkurti, "Data leakage threats and protection in split learning: a survey," in *Proceedings of the 2023 International Conference on Intelligent Computing and Its Emerging Applications*, 2023, pp. 141–147.

[7] X. Xu, M. Yang, W. Yi, Z. Li, J. Wang, H. Hu, Y. Zhuang, and Y. Liu, "A stealthy wrongdoer: Feature-oriented reconstruction attack against split learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 12 130–12 139.

[8] E. Erdoğan, A. Küpçü, and A. E. Çiçek, "Unsplit: Data-oblivious model inversion, model stealing, and label inference attacks against split learning," in *Proceedings of the 21st Workshop on Privacy in the Electronic Society*, 2022, pp. 115–124.

[9] X. Chu, L. Qiao, X. Lin, S. Xu, Y. Yang, Y. Hu, F. Wei, X. Zhang, B. Zhang, X. Wei *et al.*, "Mobilevlm: A fast, strong and open vision language assistant for mobile devices," *arXiv preprint arXiv:2312.16886*, 2023.

[10] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 615–629, 2017.

[11] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. Pmlr, 2017, pp. 1273–1282.

[12] Z. Chen, N. Lv, P. Liu, Y. Fang, K. Chen, and W. Pan, "Intrusion Detection for Wireless Edge Networks Based on Federated Learning," *IEEE Access*, vol. 8, pp. 217 463–217 472, 2020.

[13] A. Bhavani and V. Ponnusamy, "Genetic algorithm and the Kruskal-Wallis H-test-based trainer selection federated learning for IoT security," *IEEE Access*, 2024.

[14] D. Rho, T. Kim, M. Park, J. W. Kim, H. Chae, E. K. Ryu, and J. H. Cheon, "Encryption-friendly llm architecture," *arXiv preprint arXiv:2410.02486*, 2024.

[15] Z. Ji, Z. C. Lipton, and C. Elkan, "Differential privacy and machine learning: a survey and review," *arXiv preprint arXiv:1412.7584*, 2014.

[16] Y. Gao, W. Luo, C. Wang, N. S. Ahmad, X. Wang, and P. Goh, "A privacy-preserving multi-user retrieval system for multimodal artificial intelligence," *Scientific Reports*, 2026.

[17] F. De Santis, A. Schauer, and G. Sigl, "Chacha20-poly1305 authenticated encryption for high-speed embedded iot applications," in *Design, automation & test in europe conference & exhibition (DATE), 2017*. IEEE, 2017, pp. 692–697.

[18] K. D. Duy, T. Noh, S. Huh, and H. Lee, "Confidential machine learning computation in untrusted environments: A systems security perspective," *Ieee Access*, vol. 9, pp. 168 656–168 677, 2021.

[19] Z. Bai, M. Wang, F. Guo, Y. Guo, C. Cai, R. Bie, and X. Jia, "Secmdp: Towards privacy-preserving multimodal deep learning in end-edge-cloud," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 1659–1670.

[20] F. Mo, A. S. Shamsabadi, K. Katevas, S. Demetriou, I. Leontiadis, A. Cavallaro, and H. Haddadi, "Darknetz: towards model privacy at the edge using trusted execution environments," in *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*, 2020, pp. 161–174.

[21] J. Qin, X. Zhang, B. Liu, and J. Qian, "A split-federated learning and edge-cloud based efficient and privacy-preserving large-scale item recommendation model," *Journal of Cloud Computing*, vol. 12, no. 1, p. 57, 2023.

[22] A. Kembhavi, M. Salvato, E. Kolve, M. Seo, H. Hajishirzi, and A. Farhadi, "A diagram is worth a dozen images," in *European conference on computer vision*. Springer, 2016, pp. 235–251.